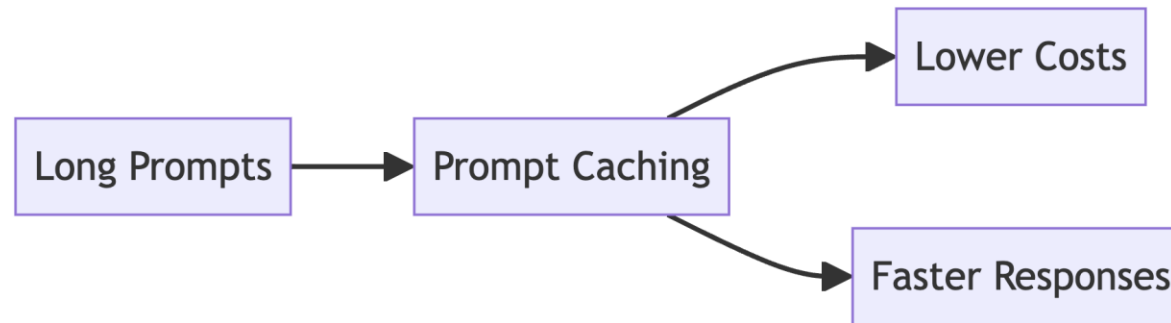
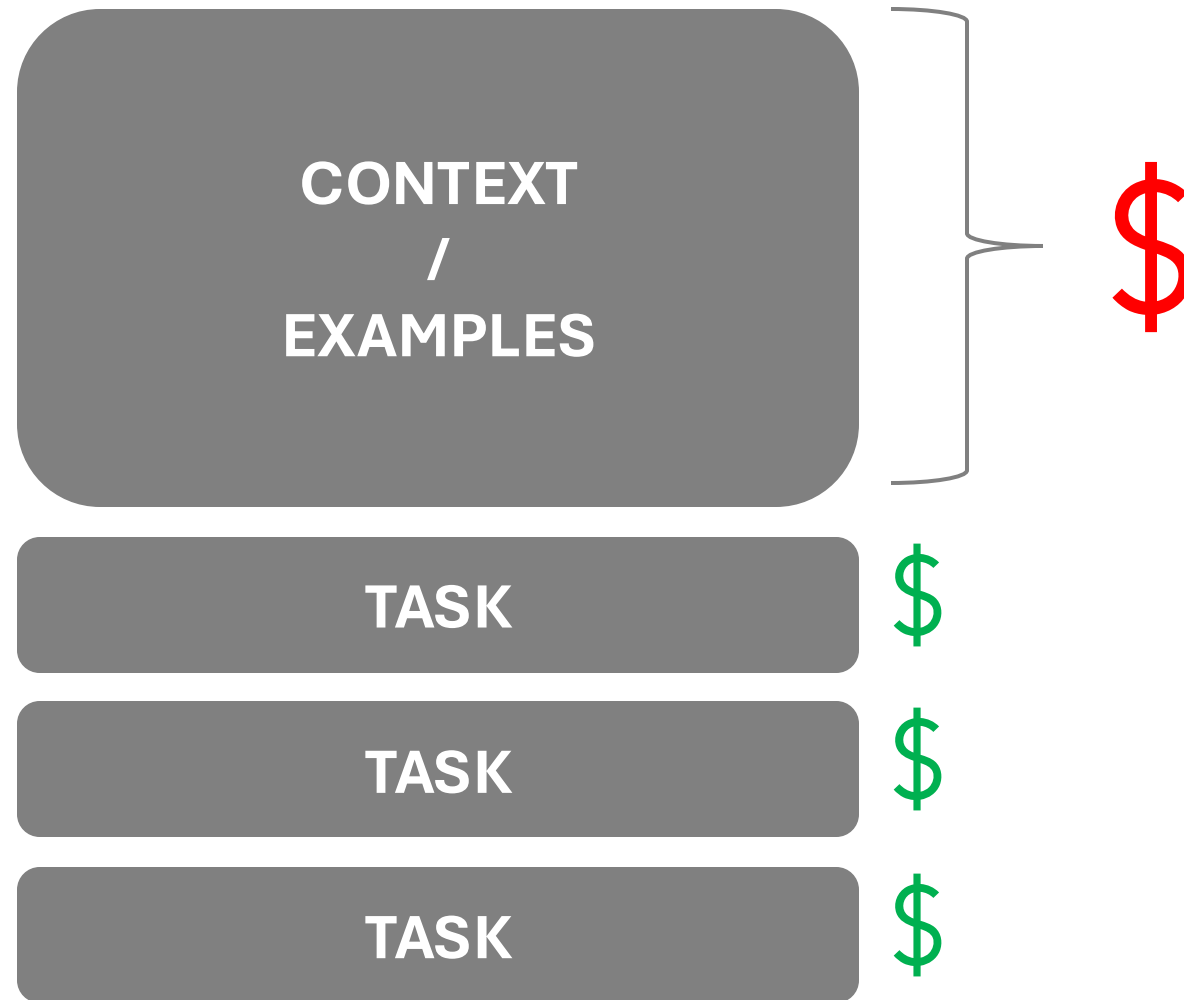


What is Prompt Caching?

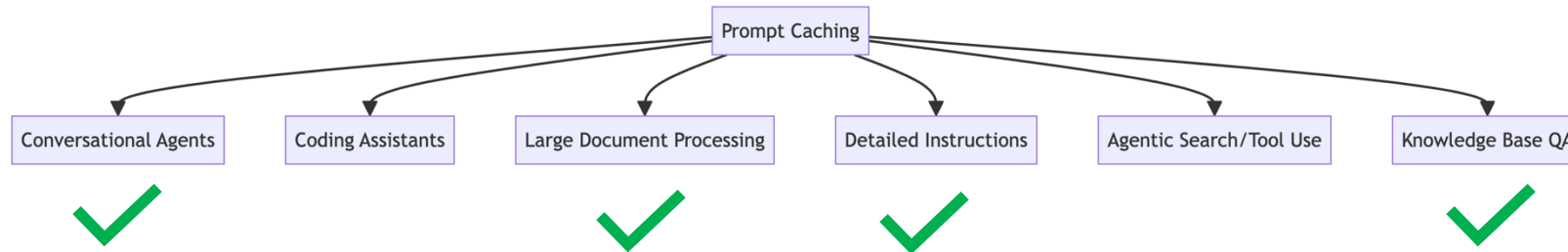


- New feature on Anthropic API for **Claude 3.5 Sonnet** and **Claude 3 Haiku**
- Caches frequently used context between API calls
- Reduces costs by **up to 90%** and **latency by up to 85%** for long prompts
- Great for *convos, coding assistants, and processing large documents*

What is Prompt Caching?



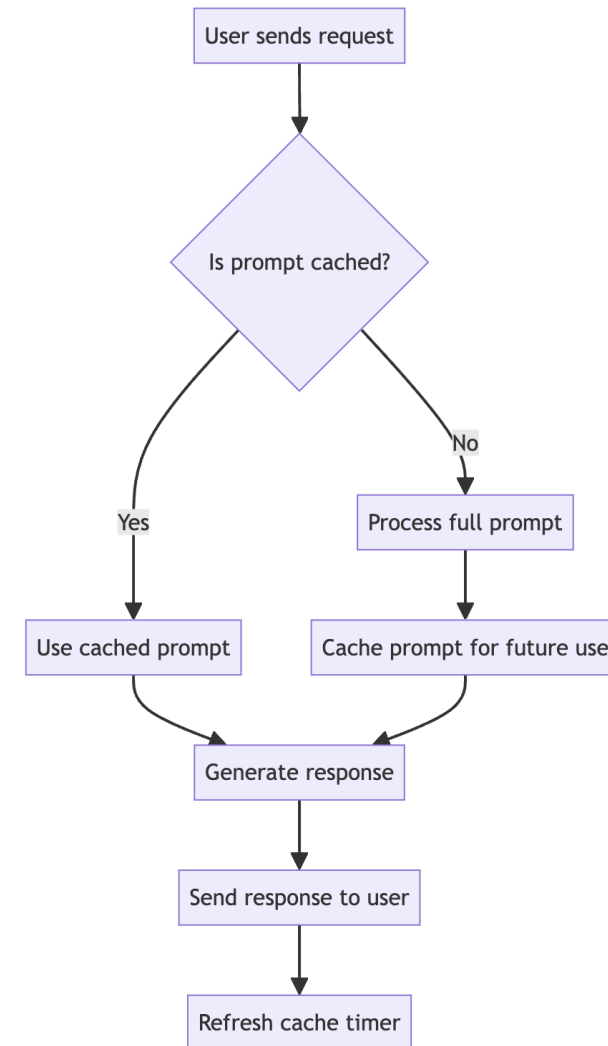
When Could You Use Prompt Caching?



- Conversational agents with **long instructions or uploaded docs**
- Coding assistants for **improved autocomplete** and codebase Q&A
- **Large document processing without increasing** response latency
- ***Detailed instruction sets with tons of examples***
- Agentic search and tool use scenarios
- Bringing knowledge bases alive (*books, papers, transcripts*)

How Does it Work?

- System checks if prompt prefix is already cached from recent query
- **Uses cached version if found**, reducing processing time and costs
- Otherwise, **processes full prompt** and caches prefix for future use
- Cache has a **5-minute lifetime**, *refreshed with each use*



BIG DISCLAIMERS!

Place all **STATIC**
instructions and
examples in cache

Minimum prompt length
should be **1024 tokens for
Sonnet, 2048 for Haiku**

No **manual cache**
clearing

Not worth it unless
large population

How Does Pricing Work?

Model	Base Input Tokens	Cache Writes	Cache Hits	Output Tokens
Claude 3.5 Sonnet	\$3 / MTok	\$3.75 / MTok	\$0.30 / MTok	\$15 / MTok
Claude 3.0 Haiku	\$0.25 / MTok	\$0.30 / MTok	\$0.03 / MTok	\$1.25 / MTok
Claude 3.0 Opus (coming soon)	\$15 / MTok	\$18.75 / MTok	\$1.50 / MTok	\$75 / MTok

- **Cache write** → **25%** more expensive than base input tokens
- **Cache read** → **90%** cheaper than base input tokens

Let's dive into the code 🤖

colab